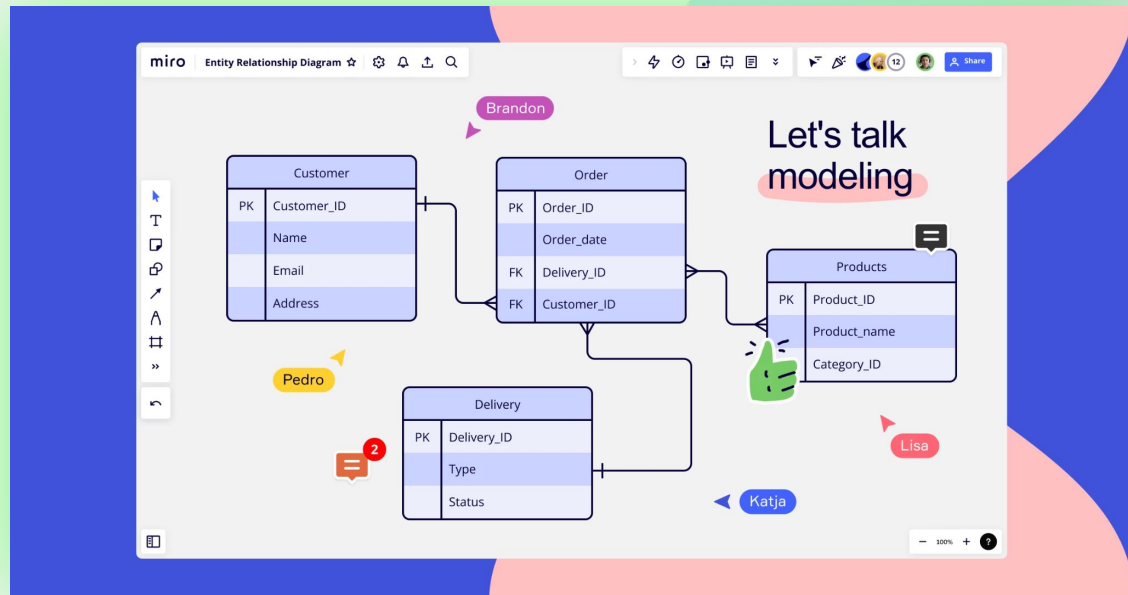


Relationships

Part II



Important aspects of a relationships

- **Purpose**: What does this relationship **mean in the real world**?
- Why do these two tables need to be connected?, What story does the relationship tell?
- Examples: A user **writes** many posts, A student **enrolls in** a class , An order **contains** products
- **If you can't describe the relationship with a verb phrase, you probably haven't modeled it correctly.**

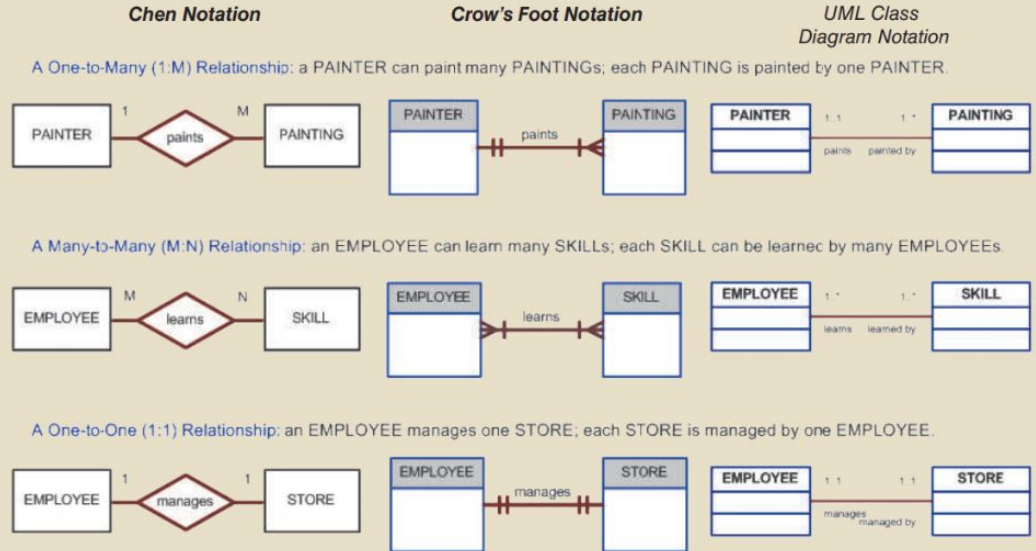
- **Multiplicity/Cardinality**: How many records on one side can be related to the other?
- One or many?, On which side?
- **Common patterns**:
 - One-to-one
 - One-to-many
 - Many-to-many
- **Example**:
- **One author → many books, One class → many students**
- **Many students ↔ many classes**
- Cardinality is about **count**, not implementation.
- Foreign keys come *after* you define this

- **Optionality**: Is the relationship **required or optional**?
- Can this record exist *without* the other one?
Is the foreign key allowed to be NULL?
- A user **may** have a profile (optional)
- A post **must** belong to a user (required)
- An order **may** have a discount (optional)
- Optionality is about **rules of the system**, not edge cases.

notation techniques

- UML (Unified Modeling Language)
- Crows Foot
- Chen

FIGURE 2.3 THE ER MODEL NOTATIONS



The **purpose** of a relationship

- **Relationships must be labelled to explain their purpose**
- Some standards have both directions of the relationship being labeled.
- Assuming you just label one direction, best practice is to:
- Use Active Voice
- Can include an Arrow to indicate the direction.



multiplicity: Describes the number of occurrences that may occur on either side of the relationship

one-to-one
one-to-many or
many-to-one
***many-to-many**

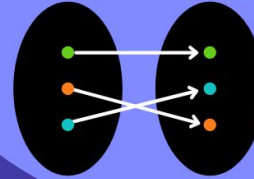
Relations

(Map each input to at least one output)

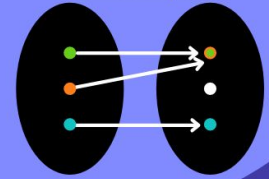
Functions

(Map each input to exactly one output)

One-to-one
(each input maps to a distinct output)

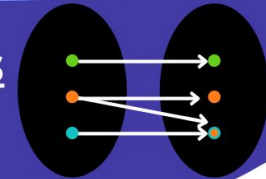


Many-to-one
(multiple inputs map to same output)



Not functions

(Map some inputs to multiple outputs)



one-to-one

- Not very common in relational databases
- Typically happen when you want to split a table because there are many blank (null) cells
- In this case, the Primary Key of both tables will be the same

employee
<u>employeeid</u>
firstName
lastName
phoneNumber
email
fullTimeFlag
salary
annualReviewDate
hourlyRate
maxWeeklyHours

employee									
<u>employeeid</u>	firstName	lastName	phoneNumber	email	fullTimeFlag	salary	annualReviewDate	hourlyRate	maxWeeklyHours
1	Tom	Brown	2125551232	tbrown@company.com	F	50000	6/1/2021		
2	John	Smith	2125635462	jsmith@company.com	F	55000	7/1/2021		
3	Fred	Jones	2127774645	fjones@company.com	P			23	15
4	Dick	Tracy	2017464872	dtracy@company.com	F	50000	6/1/2021		
5	Sally	Smith	2017484644	ssmith@company.com	P			22	20
6	Mary	Baker	2127774655	mbaker@company.com	P			30	17
7	Harry	Taylor	2038567775	htaylor@company.com	F	60000	8/1/2021		
8	Jerry	Jones	2127773564	jjones@company.com	F	50000	6/1/2021		
9	Susan	Chambers	3127774848	schambers@company.com	P			25	20
10	Julie	Brown	2127774653	jbrown@company.com	P			28	20

one-to-one

employee

employeeid

firstName

lastName

phoneNumber

email

fullTimeFlag

salary

annualReviewDate

hourlyRate

maxWeeklyHours

employee

employeeid

firstName

lastName

phoneNumber

email

fullTimeFlag

full_time_employee

employeeid

salary

annualReviewDate

part_time_employee

employeeid

hourlyRate

maxWeeklyHours

one-to-one

employee
employeeId
firstName
lastName
phoneNumber
email
fullTimeFlag
salary
annualReviewDate
hourlyRate
maxWeeklyHours

employee									
employeeId	firstName	lastName	phoneNumber	email	fullTimeFlag	salary	annualReviewDate	hourlyRate	maxWeeklyHours
1	Tom	Brown	2125551232	tbrown@company.com	F	50000	6/1/2021		
2	John	Smith	2125635462	jsmith@company.com	F	55000	7/1/2021		
3	Fred	Jones	2127774645	fjones@company.com	P			23	15
4	Dick	Tracy	2017464872	dtracy@company.com	F	50000	6/1/2021		
5	Sally	Smith	2017484644	ssmith@company.com	P			22	20
6	Mary	Baker	2127774655	mbaker@company.com	P			30	17
7	Harry	Taylor	2038567775	htaylor@company.com	F	60000	8/1/2021		
8	Jerry	Jones	2127773564	jjones@company.com	F	50000	6/1/2021		
9	Susan	Chambers	3127774848	schambers@company.com	P			25	20
10	Julie	Brown	2127774653	jbrown@company.com	P			28	20

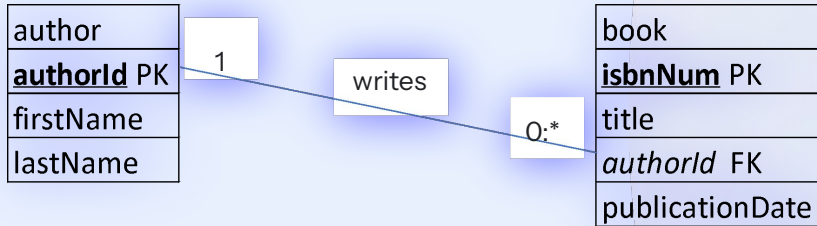
employee					
employeeId	firstName	lastName	phoneNumber	email	fullTimeFlag
1	Tom	Brown	2125551232	tbrown@company.com	F
2	John	Smith	2125635462	jsmith@company.com	F
3	Fred	Jones	2127774645	fjones@company.com	P
4	Dick	Tracy	2017464872	dtracy@company.com	F
5	Sally	Smith	2017484644	ssmith@company.com	P
6	Mary	Baker	2127774655	mbaker@company.com	P
7	Harry	Taylor	2038567775	htaylor@company.com	F
8	Jerry	Jones	2127773564	jjones@company.com	F
9	Susan	Chambers	3127774848	schambers@company.com	P
10	Julie	Brown	2127774653	jbrown@company.com	P

full_time_employee		
employeeId	salary	annualReviewDate
1	50000	6/1/2021
2	55000	7/1/2021
4	50000	6/1/2021
7	60000	8/1/2021
8	50000	6/1/2021

part_time_employee		
employeeId	hourlyRate	maxWeeklyHours
3	23	15
5	22	20
6	30	17
9	25	20
10	28	20

One-to-many relationships

- Most relationships in a relational database are one to many
- The one side has the “connecting” attribute as the Primary Key. Only one record in the table can have a specific value.
- The many side has the “connecting” attribute as the Foreign Key. There might be many records with the same value

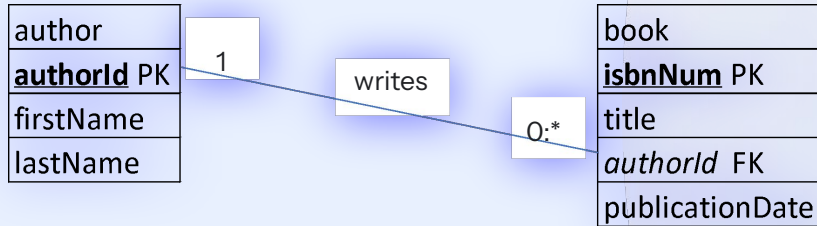


author		
<u>authorId</u>	<u>firstName</u>	<u>lastName</u>
1	William	Shakespeare
2	Charles	Dickens
3	Philip	Dick

book			
<u>isbnNum</u>	<u>title</u>	<u>authorId</u>	<u>publicationDate</u>
13-2375432	Twelfth Night	1	12/1/2010
13-6275483	David Copperfield	2	11/3/2009
13-8953736	MacBeth	1	11/20/2010
13-3457896	Do Androids Dream of Electric Sheep?	3	3/5/2009
13-5967232	Tale of Two Cities	2	4/2/2010
13-5525736	Oliver Twist	2	3/15/2010

One-to-many relationships

- They express a strong relationship, where one side is dependent upon the other.
- The relationship can be interpreted as follows:
- One author can write many books (books is the many side)
- One book can only be written by one author (author is the one side)

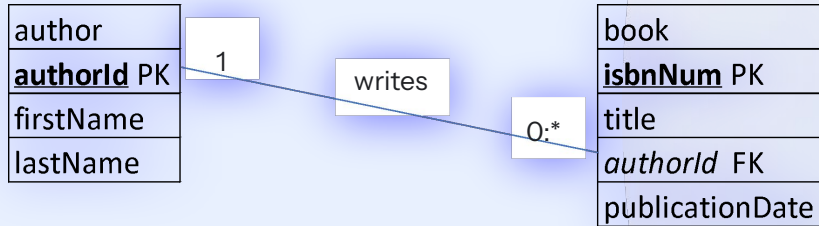


author		
authorId	firstName	lastName
1	William	Shakespeare
2	Charles	Dickens
3	Philip	Dick

book			
isbnNum	title	authorId	publicationDate
13-2375432	Twelfth Night	1	12/1/2010
13-6275483	David Copperfield	2	11/3/2009
13-8953736	MacBeth	1	11/20/2010
13-3457896	Do Androids Dream of Electric Sheep?	3	3/5/2009
13-5967232	Tale of Two Cities	2	4/2/2010
13-5525736	Oliver Twist	2	3/15/2010

One-to-many relationships

- On the many side, the “connecting attribute” describes the dependence – the author of the book
- On the one side, there cannot be an attribute from the other entity as there may be many of them.



author		
<u>authorId</u>	<u>firstName</u>	<u>lastName</u>
1	William	Shakespeare
2	Charles	Dickens
3	Philip	Dick

book			
<u>isbnNum</u>	<u>title</u>	<u>authorId</u>	<u>publicationDate</u>
13-2375432	Twelfth Night	1	12/1/2010
13-6275483	David Copperfield	2	11/3/2009
13-8953736	MacBeth	1	11/20/2010
13-3457896	Do Androids Dream of Electric Sheep?	3	3/5/2009
13-5967232	Tale of Two Cities	2	4/2/2010
13-5525736	Oliver Twist	2	3/15/2010

many-to-many relationships

- Many to many relationships cannot be implemented directly in a relational database.
- These express a weak relationship between two independent entities – neither entity is dependent upon the other.

customer

<u>customerId</u> PK
firstName
lastName
age

book

<u>isbnNum</u> PK
title
authorId FK
publicationDate

*

*

customer

customerId	firstName	lastName	age
1	Tom	Brown	24
2	John	Smith	32
3	Fred	Jones	35
4	Dick	Tracy	25
5	Sally	Smith	41

book

<u>isbnNum</u>	title	authorId	publicationDate
13-2273645	David Copperfield	2	2/10/2009
11-3736363	Macbeth	1	3/1/2010
14-2727277	Twelfth Night	1	4/3/2009
14-2534344	1984	4	6/1/2010

many-to-many relationships

- They can be recognized because the relationships in both directions include many:
- One customer can borrow many books
- One book can be borrowed by many customer
- Because the entities are independent, you can't include the other as a Foreign Key



customer

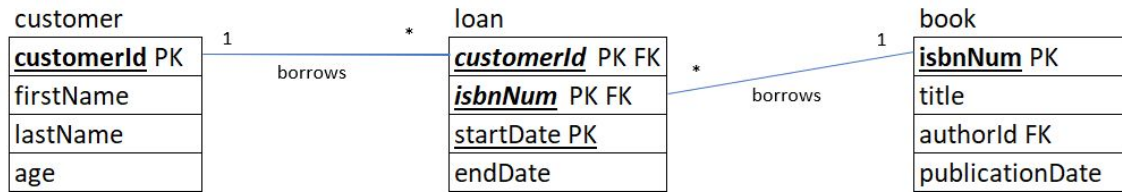
customerId	firstName	lastName	age
1	Tom	Brown	24
2	John	Smith	32
3	Fred	Jones	35
4	Dick	Tracy	25
5	Sally	Smith	41

book

isbnNum	title	authorId	publicationDate
13-2273645	David Copperfield	2	2/10/2009
11-3736363	Macbeth	1	3/1/2010
14-2727277	Twelfth Night	1	4/3/2009
14-2534344	1984	4	6/1/2010

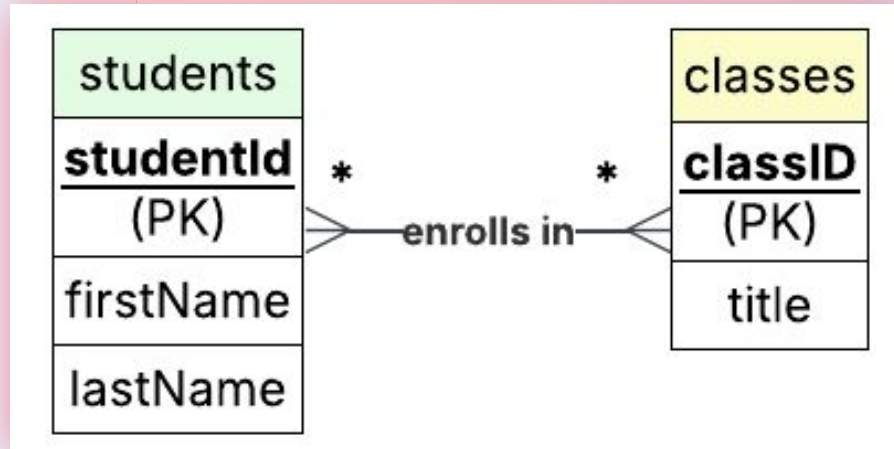
many-to-many relationships

- In Relational Databases, many to many relationships are implemented using “bridging” or “linking” tables
- They convert one Many to Many relationship into two One to Many relationships
- The new relationships conform to all the One to Many relationship rules.



many-to-many relationships

- “How many students are in a class?” → Many
- “How many classes can a student take?” → Many
- “How many values can a column store?” → One
- We can't put `studentId` in `classes` because one class has many students, and a single foreign key column can only store one value per row.



many-to-many relationships

- “How many students are in a class?” → Many
- “How many classes can a student take?” → Many
- “How many values can a column store?” → One
- We can't put `studentId` in `classes` because one class has many students, and a single foreign key column can only store one value per row.

<code>studentId</code>	<code>firstName</code>	<code>lastName</code>
------------------------	------------------------	-----------------------

1	Alice	Nguyen
2	Bob	Martinez
3	Carol	Smith

<code>classID</code>	<code>title</code>
----------------------	--------------------

101	Database Design
102	Web Development
103	Data Visualization

<code>classID</code>	<code>title</code>	<code>studentId</code>
----------------------	--------------------	------------------------

101	Database Design	1, 2
102	Web Development	1, 3
103	Data Visualization	2

We can't have multiple values in `studentId` like that

many-to-many relationships

studentId | firstName | lastName

1	Alice	Nguyen
2	Bob	Martinez
3	Carol	Smith

students
<u>studentId</u> (PK)
firstName
lastName

classID | title

101	Database Design
102	Web Development
103	Data Visualization

classes

<u>classID</u> (PK)
title

starts

enrollment
<u>studentId</u> (PK, FK(students.studentId))
<u>classID</u> (PK,FK (class.classID))
semester
grade

registers in

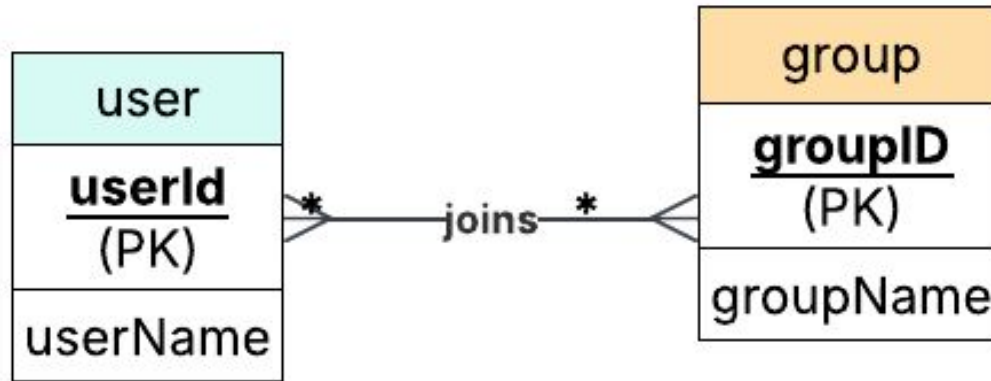
studentId | classID | semester | grade

1	101	Fall 2025	A
1	102	Fall 2025	B+
2	101	Fall 2025	B
2	103	Fall 2025	A-
3	102	Fall 2025	A

- “What is actually happening between a student and a class?”
- **Enrollment**
- Enrollment is:
- An event
- An interaction
- Something that can have its own data
- To prevent duplicate enrollments, we use a composite primary key: (studentId, classID)

many-to-many relationships

- “How many users are in a group?” → Many
- “How many groups can a user join?” → Many
- “How many values can a column store?” → One
- Can we put **groupID** in **user**? No — one user joins many groups.
- Can we put **userId** in **group**? No — one group has many users.
- A single foreign key column can only store **one value per row**.



many-to-many relationships

- “How many users are in a group?” → Many
- “How many groups can a user join?” → Many
- “How many values can a column store?” → One
- Can we put `groupID` in `user`? No — one user joins many groups.
- Can we put `userId` in `group`? No — one group has many users.
- A single foreign key column can only store **one value per row**.

userId	userName
1	alice
2	bob
3	carol

groupID	groupName
10	admins
20	editors
30	viewers

userId	groupID
1	10, 20

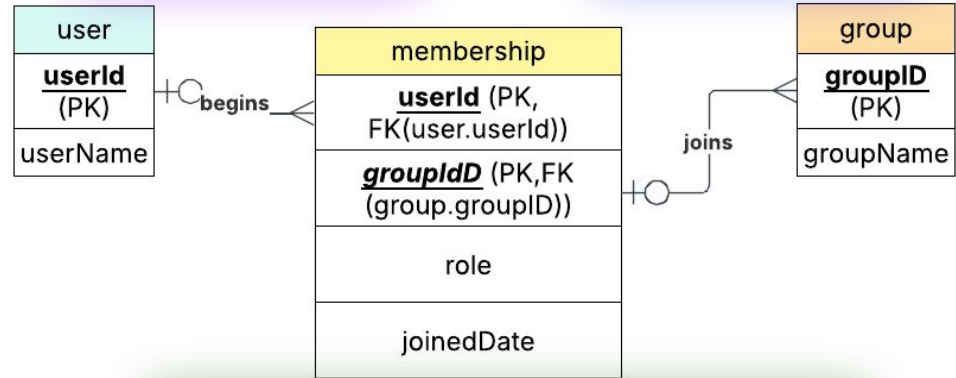
We can't have multiple values for groupID like that

many-to-many relationships

userId	userName
1	alice
2	bob
3	carol

groupID	groupName
10	admins
20	editors
30	viewers

- “What is actually happening between a user and a group?”
- **Joining or membership**
- Membership is:
- An event
- An interaction
- Something that can have its own data
- We create a new table to represent enrollment.
- Each row means:
- “This user joined this group.”
- To prevent duplicate memberships, we use a composite primary key: **(userId, groupID)**



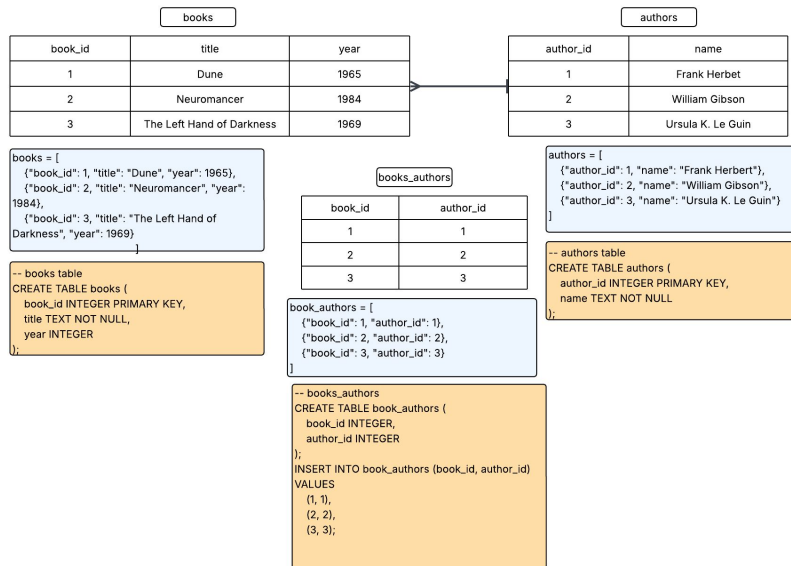
userId	groupID	role	joinedDate
1	10	owner	2024-01-15
1	20	editor	2024-02-01
2	20	editor	2024-02-10
2	30	viewer	2024-02-10
3	30	viewer	2024-03-05

Optionality

- Some relationships are mandatory. You could say that the dependent entity cannot exist without its master.
- Some relationships are optional.
- This explains the multiplicity notation:
- 0:1 as opposed to 1:1 or 0:* as opposed to 1:*
- The zeroes mean that it is OK if there are none.
- A customer can borrow none to many books. For example, when someone new joins the library, they must be set up in the database before they have an opportunity to borrow any books.

Relationships are built through assumptions

- Not only is it OK for you to state your assumptions about a model, but it is also a very smart thing to do
- For example, you can assume that a book is only written by one author.



Exercise 1

The following diagram shows two tables that are part of a bank database. What is the multiplicity of the relationship between these two tables? If it is one to many, indicate which is the one side and which the many side. If there is anything missing from these tables that is required to implement the relationship, please indicate what that is.

customer

customerId
firstName
lastName
address

account

accountNumber
accountType
currentBalance
customerId

Exercise 2

The following diagram shows two tables that are part of a sports database. What is the multiplicity of the relationship between these two tables? If it is one to many, indicate which is the one side and which the many side. If there is anything missing from these tables that is required to implement the relationship, please indicate what that is.

player

playerId
firstName
lastname
position
age

team

teamId
teamName
location

Exercise 3

The following diagram shows two tables that are part of a store database. What is the multiplicity of the relationship between these two tables? If there is anything missing from these tables that is required to implement the relationship, please indicate what that is. If a bridge table is required what would be the foreign and primary keys?

order

orderId
customerId
date
totalDollars

products

productId
name
price